

SISTEMAS PROGRAMABLES

Semáforo Vehicular y Peatonal

Reporte de práctica

Integrantes del equipo:

Daniel Bustamante

Iván Luan

Materia: Sistemas Programables

Plataforma: Arduino UNO R4 WiFi

1. Introducción

Un semáforo vehicular y peatonal es un sistema de control ampliamente utilizado como caso de estudio en sistemas programables porque combina temporización, entradas de usuario y múltiples salidas que deben coordinarse de forma segura. En esta práctica se implementó dicho sistema en un Arduino UNO R4 WiFi utilizando una máquina de estados finitos (FSM), evitando el uso de `delay()` para que el sistema pueda atender en todo momento la solicitud de un peatón.

2. Objetivo

Diseñar e implementar un semáforo vehicular y peatonal controlado por una máquina de estados finitos, que temporee sus fases con `millis()` (sin bloquear la ejecución), y que permita a un peatón solicitar el cruce mediante un botón con antirrebote (debounce) por software.

3. Requisitos de la práctica

- El sistema no debe usar `delay()` en ningún punto del programa.
- Toda la temporización se maneja con `millis()`.
- Los estados se modelan con un enum class, evitando variables globales de tipo débil (como enteros sueltos).
- El botón peatonal solo debe tener efecto si se presiona durante el vehicular en verde o amarillo.
- Al llegar el vehicular a rojo, si existe una solicitud pendiente, debe atenderse de inmediato.
- Si nadie solicita cruzar, el ciclo debe continuar de forma automática.

4. Materiales y conexiones

Se utilizaron 5 LEDs (2 para el semáforo peatonal y 3 para el vehicular), 5 resistencias de 220–330 Ω , un pulsador y un Arduino UNO R4 WiFi. La asignación de pines fue la siguiente:

Pin	Elemento	Función
D2	LED rojo vehicular	Salida digital (OUTPUT)
D3	LED amarillo vehicular	Salida digital (OUTPUT)
D4	LED verde vehicular	Salida digital (OUTPUT)
D5	LED rojo peatonal	Salida digital (OUTPUT)
D6	LED verde peatonal	Salida digital (OUTPUT)
D7	Botón peatonal	Entrada digital (INPUT_PULLUP)

El botón se conectó entre el pin D7 y GND, configurando el pin como INPUT_PULLUP, de modo que el estado en reposo es HIGH y al presionar cambia a LOW.

5. Diseño de la máquina de estados

El sistema se modeló con cuatro estados: VEH_VERDE, VEH_AMARILLO, VEH_ROJO y PEA_VERDE. La siguiente tabla resume la duración de cada estado y el color mostrado en ambos semáforos:

Estado	Duración	Vehicular	Peatonal
VEH_VERDE	8 s	Verde	Rojo
VEH_AMARILLO	2 s	Amarillo	Rojo
VEH_ROJO	6 s (o hasta solicitud)	Rojo	Rojo
PEA_VERDE	5 s	Rojo	Verde

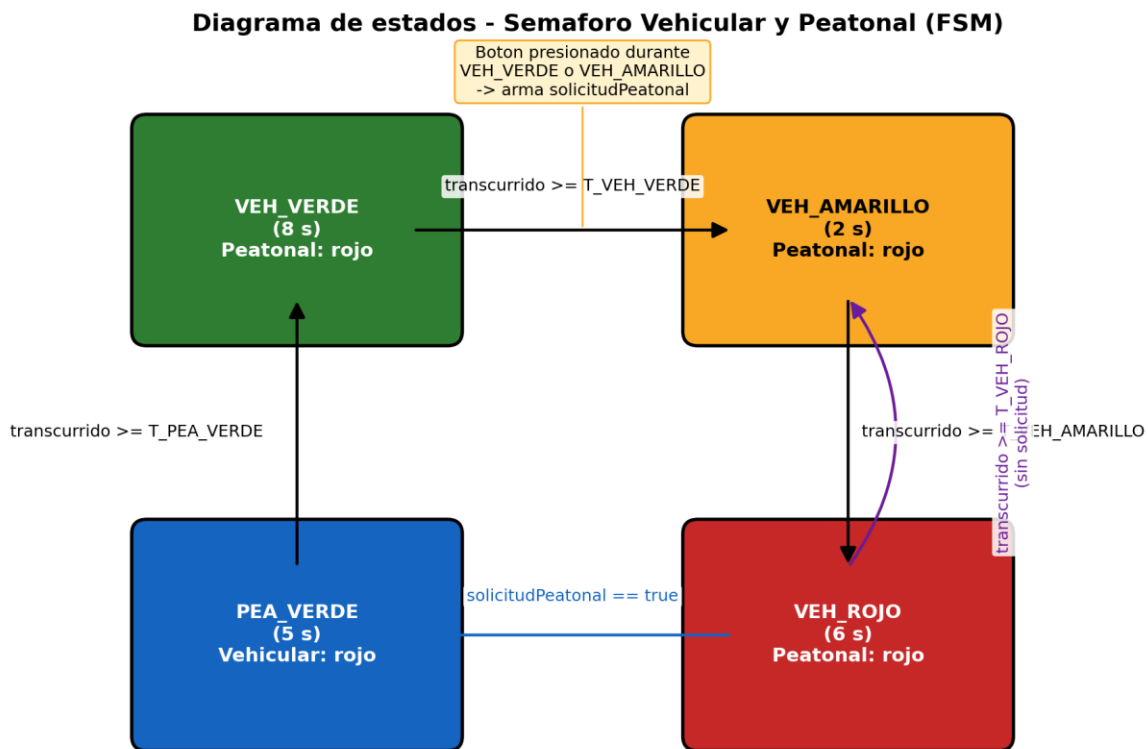


Figura 1. Diagrama de transición de estados del semáforo.

Las transiciones VEH_VERDE → VEH_AMARILLO → VEH_ROJO ocurren automáticamente al cumplirse cada temporizador. Desde VEH_ROJO existen dos caminos posibles: si hay una solicitud peatonal

pendiente, se pasa de inmediato a PEA_VERDE; si no la hay, se espera el tiempo T_VEH_ROJO y se regresa a VEH_VERDE, reiniciando el ciclo.

6. Implementación

6.1 Antirrebote (debounce) del botón

Para evitar lecturas falsas por rebote mecánico del pulsador, se separaron dos variables: la última lectura física del pin (para detectar cambios) y el último estado ya confirmado como estable durante al menos 50 ms. Solo cuando la señal permanece estable ese tiempo se considera una pulsación válida:

```
bool botonPresionado() {
    int lectura = digitalRead(PIN_BOTON);
    bool presionDetectada = false;

    if (lectura != ultimaLecturaCruda) {
        ultimoCambioLectura = millis();
    }

    if ((millis() - ultimoCambioLectura) > T_DEBOUNCE) {
        if (lectura != estadoEstable) {
            estadoEstable = lectura;
            if (estadoEstable == LOW) { // INPUT_PULLUP: LOW = presionado
                presionDetectada = true;
            }
        }
    }

    ultimaLecturaCruda = lectura;
    return presionDetectada;
}
```

6.2 Lógica principal de la máquina de estados

El ciclo principal calcula el tiempo transcurrido en el estado actual ($\text{transcurrido} = \text{millis()} - \text{inicioEstado}$) y decide, según el estado y las condiciones, si debe transicionar. La función `cambiarEstado()` centraliza el cambio de estado, el reinicio del temporizador y la actualización de los LEDs:

```
enum class EstadoSemaforo {
    VEH_VERDE,
    VEH_AMARILLO,
    VEH_ROJO,
    PEA_VERDE
};

EstadoSemaforo estadoActual = EstadoSemaforo::VEH_VERDE;
unsigned long inicioEstado = 0;
bool solicitudPeatonal = false;

void cambiarEstado(EstadoSemaforo nuevoEstado) {
    estadoActual = nuevoEstado;
    inicioEstado = millis();
}
```

```

    aplicarSalidasEstado(nuevoEstado);
}

void loop() {
    unsigned long transcurrido = millis() - inicioEstado;

    if (botonPresionado()) {
        if (estadoActual == EstadoSemaforo::VEH_VERDE ||
            estadoActual == EstadoSemaforo::VEH_AMARILLO) {
            solicitudPeatonal = true;
        }
    }

    switch (estadoActual) {
        case EstadoSemaforo::VEH_VERDE:
            if (transcurrido >= T_VEH_VERDE) cambiarEstado(EstadoSemaforo::VEH_AMARILLO);
            break;

        case EstadoSemaforo::VEH_AMARILLO:
            if (transcurrido >= T_VEH_AMARILLO) cambiarEstado(EstadoSemaforo::VEH_ROJO);
            break;

        case EstadoSemaforo::VEH_ROJO:
            if (solicitudPeatonal) {
                solicitudPeatonal = false;
                cambiarEstado(EstadoSemaforo::PEA_VERDE);
            } else if (transcurrido >= T_VEH_ROJO) {
                cambiarEstado(EstadoSemaforo::VEH_VERDE);
            }
            break;

        case EstadoSemaforo::PEA_VERDE:
            if (transcurrido >= T_PEA_VERDE) cambiarEstado(EstadoSemaforo::VEH_VERDE);
            break;
    }
}

```

7. Resultados

El sistema operó de acuerdo con lo esperado: en condiciones normales, el semáforo vehicular cicla automáticamente entre verde, amarillo y rojo mientras el peatonal permanece en rojo. Al presionar el botón durante el verde o el amarillo vehicular, la solicitud queda "armada" y se atiende justo al llegar el vehicular a rojo, mostrando el peatonal en verde durante 5 segundos antes de reiniciar el ciclo. Las pulsaciones realizadas en otros momentos (por ejemplo, durante el rojo o el propio verde peatonal) se ignoran correctamente, cumpliendo el requisito de la práctica.

8. Conclusiones

La implementación mediante una máquina de estados finitos permitió estructurar el problema de forma clara y escalable: cada estado agrupa su propia lógica de salidas y condiciones de transición, lo que facilita agregar nuevos estados en el futuro (por ejemplo, un modo de parpadeo intermitente nocturno). El uso de `millis()` en lugar de `delay()` fue fundamental para poder atender el botón peatonal en cualquier

momento sin perder pulsaciones, y el antirrebote por software garantizó que cada pulsación física se registrara una sola vez.

Como equipo, esta práctica nos permitió comprender de forma aplicada los conceptos de máquinas de estados, temporización no bloqueante y manejo de entradas digitales, herramientas fundamentales en el desarrollo de sistemas programables.